11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb
```
# 3 == 3.0:
expect(3).to eq(3.0)

# ...but 3.eql?(3.0) is false:
expect(3).not_to eql(3.0)
```

This behavior allows 3 and 3.0 to be used as different keys in the same hash.

When in Doubt, Use eq



All these different ways to compare objects can seem confusing. When you're not sure which matcher is right, try eq first. In most situations, value equality is the one you need.

Variations

All three of these matchers have aliases that read better in composed matcher expressions:

- an_object_eq_to aliases eq
- an_object_equal_to aliases equal
- an_object_eql_to aliases eql

For instance, consider the following expectation that checks a list of Ruby classes:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb
```
expect([String, Regexp]).to include(String)
```

The intent was to require the actual Ruby String class to be present. But this spec will incorrectly allow plain Ruby strings to pass as well:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb
```
expect(['a string', Regexp]).to include(String)
```

As we saw in the previous chapter, higher-order matchers like include check their arguments with the three-quals operator, ===. In this case, RSpec ends up checking String === 'a string', which returns true.

The fix is to pass the an_object_eq_to matcher into include, to make the pass/fail criteria more precise:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb
```
expect([String, Regexp]).to include(an_object_eq_to String)
```

The three equality matchers we've discussed will work on any Ruby object.